Constructor

- A constructor is a special method used to initialize objects.
- Same name as class.
- No returnType, as it doesn't return values.
- NOT explicitly called; automatically called when object is created.
- If no constructor is defined, JVM creates a default constructor which initializes the members with their default values.

Types of Constructors:

(i) Default Constructor
(ii) Parameterized Constructor
(iii) Copy Constructor

(i) Default Constructor

- A constructor that has no parameters is called a default constructor.

(ii) Parameterized Constructor

- A constructor that has parameters is called a parameterized constructor.

Constructor Overloading

- We can have multiple constructors with different parameter list.
- JVM will not create a default constructor if any constructor is defined.

Constructor Chaining

- One constructor can call another constructor using "this" keyword inside the same class.

```
e.g.- class Temp {
        Temp() {
            this(5);   // calls constructor 2
            sout("__");
        }
        Temp(int x) {
            this(x,5); // calls constructor 3
            sout("__");
        }
        Temp(int x, int y) {
            sout("__");
        }
}
```

(iii) Copy Constructor

- A constructor used to create a new object as a copy of an existing object.

```
e.g.- Person person1 = new Person();
      Person person2 = new Person(person1);
```

- Purpose:
  - Create a duplicate
  - Create a deep copy

- Shallow Copy — copied reference points to same object
- Deep Copy — copied reference points to a newly created copy of object.
- Java is NOT a fully object-oriented programming language because primitives like int, float exist

Wrapper Class

- used to convert primitive data types to objects
  - e.g. - "Character" for char, "Integer" for int, "Float" for float, etc.

- Features:

(i) - Autoboxing — automatic conversion of primitive types to objects of their corresponding wrapper class

- Unboxing — converting a wrapper object back to its corresponding primitive type.

```
e.g.- int x = 10;
      Integer obj = x;   // Autoboxing (introduced in Java 9)
      int y = obj;       // Unboxing
      Integer obj2 = Integer.valueOf(x);
      Integer obj3 = new Integer(x); // Correct but deprecated since Java 9
```

(ii) Useful methods to manipulate and convert data

```
e.g.- Integer.parseInt("123"); // converts String to Integer   } Static Methods
      Double.toString(10.5);   // converts Double to String
```

(iii) Immutable Objects, i.e. values can't be changed after creation

```
e.g.- Integer a = 10;
      a = 20; // Error
```

(iv) Used in collections like ArrayList as they only accept objects

```
e.g.- ArrayList<Integer> a = new ArrayList<>();
                  int X
            (only supports classes)
      a.add(10); // autoboxed to Integer
      a.add(20); // We can keep adding more elements without giving a fixed size
      HashMap<String, Double> hmap = new HashMap<>();
      hmap.put("Apple", 1.99); // 1.99 autoboxed to Double
      hmap.keySet(); // returns array of the keys (with unboxing if applicable)
```

Arrays

- Arrays are collection of homogeneous data stored contiguously in memory
- Unlike C/C++, arrays are objects in Java.
- In Java, all arrays are dynamically allocated (stored in heap)

e.g.- `int[] a;` (or) `int a[];` // Declaration

`int[] a = new int[5];` // Declaration + Creation of array of size 5 with elements initialized with 0

`a.length;` // to get length of array

Class of Array

```
int[] arr = new int[6];
arr.getClass().getName();
```
↳ O/P: [I → 1D array of integers (JVM representation)

`arr.instanceOf(Object);`
↳ O/P: true (all classes in Java inherit from "Object" class)

Array Type	JVM Representation
int[]	[I
double[]	[D
boolean[]	[Z
char[]	[C
String[]	[Ljava.lang.String
int[][]	[[I

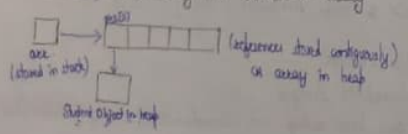
Types of Arrays

i) 1-D array

ii) Multi-Dimensional arrays

Array of Objects

```
class Student{
    ---
}
Student[] arr; //declaration of a Student array
arr = new Student[5] // memory allocation for 5 objects of Student class (create an array of Student reference)
arr[0] = new Student(); // initializing first element of array
```

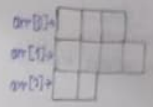


Jagged Array

- A Jagged array is an array of arrays such that member arrays can be of different sizes.

e.g.- `int arr[][] = new int[3][];`

```
arr[0] = new int[3];
arr[1] = new int[4];
arr[2] = new int[2];
```



Inheritance

Syntax: `class derivedClass extends baseClass{ ... }`

- The derived class will inherit all attributes and methods from base class.
  (subclass) (superclass)

Access Modifiers for Inheritance

i) private — not inheritable or accessible outside the class it is defined

ii) default (package-private) — accessible within same package; NOT in other packages even if inherited

iii) protected — accessible within the same package + by subclasses even in different packages (only in subclasses through inheritance)

iv) public — accessible to all — same or different package, subclass or not.

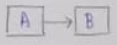
- We have to import the class in other package to use public or protected members.

e.g.-
```
package packageA;
public class Animal{
    protected String name;
    protected void display(){
        ---
    }
}
```

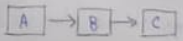
```
package packageB;
import packageA.Animal;
public class Dog extends Animal{
    --- // has access to name
    --- // has access to display()
    ---
}
```

Types of Inheritance

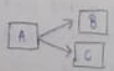
i) Single Inheritance
- when a class extends only one parent



ii) Multi-level Inheritance
- chain of inheritance



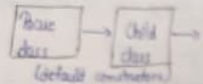
iii) Hierarchical Inheritance
- two or more classes inherit a single class



iv) Multiple Inheritance
- a class can inherit from more than one superclass (NOT supported in Java)



Order of Execution of Constructors



- In Java, we can achieve multiple inheritances only through "interfaces".

Super keyword

- If we have same variable in parent and child class, the child class one will be preferred.

Uses:

i) Calls parameterized constructor of base class (should be first line of child constructor)

ii) Access members of base class (if same name members are also in child class)

- super.variableName
- super.methodName()

Final keyword

- "final" keyword is a modifier that can be applied to variables, methods and classes to indicate they can't be changed or overridden.
- final variables can't be changed after initialization (named constants)
- final methods can't be overridden in the child classes.
- final classes can't be inherited.

Polymorphism

- "many forms" — ability of an object to take on multiple forms

Types:

i) Compile-time polymorphism — resolution of method call to its def" at compile time (early binding)
e.g. – method overloading

ii) Run-time polymorphism — resolution of method call to its def" at runtime (late binding)
e.g. – method overriding
- a.k.a "Dynamic Method Dispatch"

Method Overriding

- It allows a subclass to provide a specific implementation of a method that is already given in one of its superclasses (instance method)

- Conditions:

i) Inheritance should be there

ii) Method name, parameter list and return type should be same (or covariant return type)

iii) Access level of overriding method can't be more restrictive than overridden method.

public < protected < default < private

restrictive →

- @Override annotation can be used to help the compiler identify mistakes, like incorrect method signatures (NOT compulsory)
- Static, Final or Private methods can't be overridden.
  - Static → bound at compile-time → can't be inherited
  - Final → can't be inherited but NOT overridden

Method Hiding

- When a static method with same signature as static method in base class, it is "method hiding".

	Superclass Instance Method	Superclass Static Method
Subclass Instance Method	Overrides	Compilation Error
Subclass Static Method	Compilation Error	Hides

Dynamic Method Dispatch

- Used to impose run-time binding

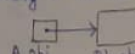
```
class A{
    void common(){
        S.O.P("Common Method");   // from A
    }
    void f1(){
        S.O.P("I'am f1");
    }
}
class B extends A{
    void common(){
        S.O.P("Common Method from B");
    }
    void f2(){
        S.O.P("I am f2");
    }
}
```

A → f1()
↓
B → f1()

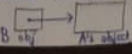
```
B obj = new B();
obj.f1();   → Compile-Time Binding
```

```
class Test{
    PSVM(String s[]){
        A obj = new B(); // Upcasting
        obj.common(); // O/P: B's o/p
        obj.f1(); // I am f1 (compile-time binding)
        obj.f2(); // ERROR (since f2() is not in A)
        B obj1 = (B) obj; // Explicit casting (since object is of B type)
        obj1.f2(); // f2
    }
}
```

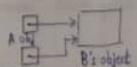
- Upcasting: reference of a base class points to child object
  - late binding or runtime binding for overridden methods



- Downcasting: reference of child class points to parent object
  - NOT allowed in Java (not automatically)

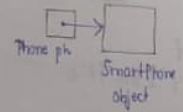


XXX (can be done explicitly)



```
class Phone{
    void call(){
        S.O.P("Calling");
    }
    void alarm(){
        SOP("Alarm at 6am");
    }
}
class Smartphone extends Phone{
    void call(){
        S.O.P("Calling from smartphone");
    }
    void clickPhoto(){
        S.O.P("Clicking Photo");
    }
}
class Test{
    P.S.V.M(String s[]){
        User Ramesh = new User(new Smartphone());
        Ramesh.useCall(); // Calling from Smartphone
        Ramesh.useAlarm(); // Alarm at 6am
    }
}
```

```
class User{
    Phone ph;
    User(Phone ph){
        this.ph=ph;
    }
    void useCall(){
        ph.call();
    }
    void useAlarm(){
        ph.alarm();
    }
}
```



- Dynamic Method Dispatch only works for instance methods
- Variables aren't overridden, as they are bound at compile time

```
class Phone{
    String model="Nokia";
    ==
}
class Smartphone{
    String model="Samsung";
    ==
}
P.S.V.M(String s[]){
    S.O.P
    Phone ph=new Smartphone();
    S.O.P(ph.model); // Nokia (bound at compile-time through Phone's reference)
}
```

Abstract Class

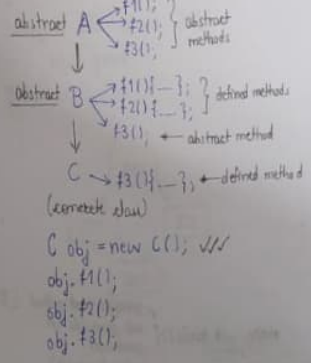
- Abstract Method: needn't be defined in the same class

```
abstract class A{
    abstract void f1();
}
class Test{
    P.S.V.M(String s[]){
        A obj = new A();
        obj.f1();
    }
}
```
} NOT allowed

- If a class has an abstract method, the class must be abstract
- Abstract class: object can't be created of this class

```
class B extends A{
    void f1(){
        ----
        ----
    }
}
```

- Abstract classes can have, instance variables, instance methods, static variables, static methods, abstract methods, constructors, final methods.
- Abstract methods can't be final.
- Abstract classes needn't include any abstract methods.
- We must implement all abstract methods in some sub-class, if any sub-class has even one abstract method it must also be made an abstract class



Interface

Syntax: interface interfaceName{

...

}

- All methods in an interface are abstract by default and also public

```
interface I1{
    void f1();   } public + abstract
    void f2();   }
    int a = 10;  } public + static + final
}
```

- Methods: public and abstract
- Variables: public, static, final

```
class Test implements I1{
    void f1(){...}
    void f2(){...}
}
```

- The implementing class must define all the abstract methods of the interface

e.g. –
```
Interface I1{
    int i = 10;
    void display();
}
class A implements I1{
    public void display(){
        S.O.P("Hello");
    }
    P.S.V.M(String s[]){
        A obj = new A();
        obj.display();   // Hello
        S.O.P(obj.i);    // 10
    }
}
```

- default method:
```
interface I1{
    void f1();
    default void f4(){ ← no need to be defined in subclasses necessarily
    }
}
```

```
static void details(){ ← interface-level method (I1.details) not accessible in subclasses (not inheritable)
}
```

- Interface can extend another interface

```
interface I1{
    void f1();
    void f2();
}
interface I2 extends I1{
    void f3();
    void f4();
}
```

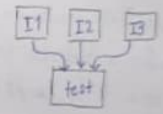
```
class Test implements I2{
    f1(){...}
    f2(){...}
    f3(){...}
    f4(){...}
}
```
} class must implement all the methods

Multiple Inheritance through Interfaces

- A class can't extend more than one other class, but can implement any no. of interfaces

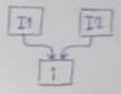
i) class test implements I1,I2,I3{

}

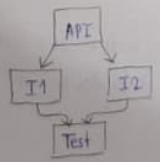


ii) interface i extends I1,I2{

}



e.g. –
```
interface API{
    default void show(){
        S.O.P("Show Method");
    }
}
interface I1 extends API{
    void display();
}
interface I2 extends API{
    void print();
}
```




```
class Test implements I1,I2{
    public void display(){
        S.O.P("display method of I1");
    }
    public void print(){
        S.O.P("print method of I2");
    }
    P.S.V.M(String s[]){
        Test obj = new Test();
        obj.show();    // Show Method
        obj.display(); // display method of I1
        obj.print();   // print method of I2
    }
}
```

Exception Handling

- Exception is an unwanted/unexpected event which occurs during execution of a program, i.e run-time, that disrupts the normal flow of the program's instructions.

eg- `int a=10, b=0;`
`int r = a/b; // This line throws an ArithmeticException`
`S.O.P(r); // Not executed`
O/P: java.lang.ArithmeticException: / by zero

try-catch

- try-block: contains the lines that can throw an exception
- catch-block: only executed if an exception occurs in try-block
- When an exception occurs, JVM creates an object of that exception class (defined in java.lang)
- Each catch block handles a particular kind of exception; when JVM throws the exception object from the try block to be caught by the catch block

Reasons for Exceptions

- Invalid User Input
- Device Failure
- Loss of Network Connection
- Physical Limitations (out of disk memory)
- Code Errors
- Out of Bound
- Null Reference
- Type Mismatch
- Opening an unavailable file
- Database Errors
- Arithmetic Errors

- throw: used to manually throw an exception
- throws: any exception thrown out of a method must be specified as such by this clause

finally block

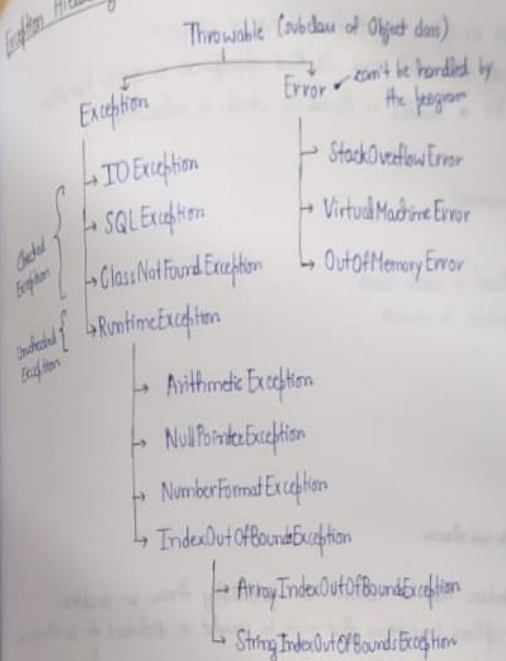
- any code that must execute after a try block completes (whether except occurs or not)
- this always runs — whether there is exception or not, whether it is handled or not

try → catch → finally

- Only try-block without catch-block causes compile-time error.
- try-block with finally-block (without catch-block) still works.

try{...} ✗ try{...} finally{...} ✓

Exception Hierarchy



- Checked Exception: compiler checks at compile-time whether these are handled or not
- Unchecked Exception: runtime-exception (program compiles successfully)

- Only one catch block is executed; when an exception is caught, the subsequent catch blocks are skipped
- More specific catch blocks should be placed before more general ones to avoid compilation errors.
  - O/P: exception ArithmeticException has already been caught

Displaying details of Exception

- The toString() method displays exception info in the format:
  Name of Exception : description of Exception
- getMessage() → only description of Exception
- printStackTrace()

How JVM handles Exception?

i) Exception Occurrence : JVM creates exception object of that type of exception

ii) Stack Unwinding : JVM unwinds the call stack to find appropriate exception handler.

- checks from child to parent till a handler is found or stack is exhausted.

iii) Finding a matching catch block

iv) "finally" block is executed (if present)

v) Propagating Uncaught Exceptions

- if it reaches main method without a catch block:
  - the Exception Stack trace is printed to console
  - the program terminates

throw

- Used to throw an exception

Syntax: `throw ThrowableInstance;`

(ThrowableInstance: of class Throwable or its sub-classes)

throws

- Used in a method signature to declare that a method can potentially throw an exception.
- Commonly used with checked exceptions (exceptions that must be caught or declared to be thrown)

↓ parent must handle it all

finally

